

Sets

1. What are sets (and documentation about sets)

Documentation

Sets are one of Python's *Built-in Types*

It's a compound data types, since it contains items of other data types (similar to Lists and Tuples)

Documentation

You find detailed the documentation here among other built-in types

https:

`//python.readthedocs.io/en/latest/library/index.html`

Then choose: Set Types - set frozenset

To reach: `https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset`

- Built-in Types
 - Truth Value Testing
 - Boolean Operations — and, or, not
 - Comparisons
 - Numeric Types — int, float, complex
 - Boolean Type - bool
 - Iterator Types
 - Sequence Types — list, tuple, range
 - Text and Binary Sequence Type Methods Summary
 - Text Sequence Type — str
 - Binary Sequence Types — bytes, bytearray, memoryview
 - Set Types — set, frozenset
 - Mapping Types — dict

2. Create and use a set

Create a set

Sets are **unordered**.

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}
2 print(english_vowels)
3 print(type(english_vowels))
```

```
{'y', 'i', 'o', 'a', 'e', 'u'}
```

```
<class 'set'>
```

Compare **with a list, which is ordered**

```
1 english_vowels_list = ["a", "e", "i", "o", "u", "y"]
2 print(english_vowels_list)
```

```
['a', 'e', 'i', 'o', 'u', 'y']
```

Create a set

Each item may occur only **once** in a set.

```
1 english_vowels = {"a", "a", "e", "e", "i", "o", "u", "y"}  
2 print(english_vowels)
```

{'i', 'u', 'o', 'y', 'e', 'a'}

Compare with a list, where the same item can occur many times.

```
1 english_vowels_list = ["a", "a", "e", "e", "i", "o", "u", "y"]  
2 print(english_vowels_list)
```

['a', 'a', 'e', 'e', 'i', 'o', 'u', 'y']

Create a set

Sets are **unindexed**.

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}
2 print(english_vowels[0]) # NOT POSSIBLE!!!!
```

```
print(english_vowels[0])
```

~~~~~

TypeError: 'set' object is not subscriptable

Compare with a **list**:

---

```
1 english_vowels_list = ["a", "e", "i", "o", "u", "y"]
2 print(english_vowels_list[0])
```

---

'a'

# Create a set

Create an empty set.

There is no “shorthand” as for lists (where you use brackets). You need to use the constructor for an empty set.

---

```
1 english_vowels = set()
2 print(type(english_vowels))
```

---

```
<class 'set'>
```

Compare with a list:

---

```
1 english_vowels_list_1 = []
2 english_vowels_list_2 = list()
3 print(type(english_vowels_list_1))
4 print(type(english_vowels_list_2))
```

---

```
<class 'list'>
```

```
<class 'list'>
```

# Create a set

You can create a set from a list or a tuple.

From a list (note what happens with duplicates):

---

```
1 vowels = set(['a', 'a', 'e', 'e', 'i', 'o', 'u', 'y'])  
2 print(vowels)
```

---

{'e', 'u', 'o', 'i', 'y', 'a'}

From a tuple (note what happens with duplicates):

---

```
1 vowels = set(('a', 'a', 'e', 'e', 'i', 'o', 'u', 'y'))  
2 print(vowels)
```

---

{'i', 'e', 'y', 'o', 'u', 'a'}

## Use the uniqueness

---

```
1 text = ["shall", "i", "compare", "thee", "to", "a",
2         "summer's", "day", "?", "thou", "art", "more",
3         "lovely", "and", "more", "temperate:",
4         "rough", "winds", "do", "shake", "the",
5         "darling", "buds", "of", "may", ",", "and",
6         "summers", "lease", "hath", "all",
7         "too", "short", "a", "date"]
8
9 words_unique = []
10
11 for w in text:
12     if w not in words_unique:
13         words_unique.append(w)
14 print(words_unique)
```

---

```
['shall', 'i', 'compare', 'thee', 'to', 'a', 'summer's', 'day',
 '?', 'thou', 'art', 'more', 'lovely', 'and', 'temperate:',
 'rough', 'winds', 'do', 'shake', 'the', 'darling', 'buds', 'of',
 'may', ',', 'lease', 'hath', 'all', 'too', 'short', 'date']
```

# Use the uniqueness

---

```
1 text = ["shall", "i", "compare", "thee", "to", "a",
2         "summer's", "day", "?", "thou", "art", "more",
3         "lovely", "and", "more", "temperate:",
4         "rough", "winds", "do", "shake", "the",
5         "darling", "buds", "of", "may", ",", "and",
6         "summers", "lease", "hath", "all",
7         "too", "short", "a", "date"]
8
9 words_unique = set(text)
10 print(words_unique)
```

---

```
{'shake', 'i', 'may', 'a', '?', 'thou',
'lease', 'and', 'darling', 'short',
'summer's', 'thee', 'day', 'temperate:',
'date', 'more', 'lovely', 'rough', ',',
'of', 'to', 'compare', 'buds', 'winds',
'hath', 'the', 'do', 'all', 'art',
'too', 'shall'}
```

### 3. Built-in functions

# Built-in functions applied on sets

`len()`

`min()`

`max()`

(All actually apply also on e.g. lists and tuples)

---

```
1 numbers = {2, 4, 6, 8, 10}
2
3 print(len(numbers))
4
5 print(min(numbers))
6 print(max(numbers))
```

---

5

2

10

## 4. Looping over sets

# Loops

Note that there is no fixed order for the loop

---

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}
2
3 for vowel in english_vowels:
4     print(vowel)
```

---

Run you program first time:

y  
e  
i  
a  
u  
o

Run you program second time:

a  
y  
u  
o  
i  
e

## 5. Checking if something is in a set

# in

**in** works as usual

---

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}
2
3 for ch in ["a", "b", "c"]:
4     if ch in english_vowels:
5         print(ch + " is a vowel")
6     else:
7         print(ch + " is not a vowel")
```

---

a is a vowel

b is not a vowel

c is not a vowel

# in

Before, we have used a tuple or a list, for a similar task. However, set is better, because it is faster to search in.

---

```
1 english_vowels = ("a", "e", "i", "o", "u", "y") # Before
2
3 english_vowels = {"a", "e", "i", "o", "u", "y"} # Better
4
5 for ch in ["a", "b", "c"]:
6     if ch in english_vowels:
7         print(ch + " is a vowel")
8     else:
9         print(ch + " is not a vowel")
```

---

a is a vowel

b is not a vowel

c is not a vowel

## 5. Methods that modify the set

# add()

Note that to add an element you use add()

(Not append as for a list. It's logical, since with a list you append a new element towards the end of a list. You don't do that with a set, you add the element.)

---

```
1 english_vowels = {"a", "e", "i", "o", "u"}
2
3 english_vowels.add("y")
4
5 print(english_vowels)
```

---

{'y', 'a', 'o', 'i', 'e', 'u'}

# add()

A set is mutable. The set is changed.

The method for adding just returns None

---

```
1 english_vowels = {"a", "e", "i", "o", "u"}
2
3 return_value = english_vowels.add("y")
4
5 print(english_vowels)
6
7 print(return_value)
```

---

{'o', 'y', 'e', 'a', 'i', 'u'}

None

# remove()

If you try to remove something that is not in the set you get a `KeyError`

---

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}
2
3 english_vowels.remove("y")
4 print(english_vowels)
5
6 english_vowels.remove("y")
```

---

`{'u', 'i', 'a', 'o', 'e'}`

`KeyError: 'y'`

# remove()

Also `remove` returns `None`.

---

```
1 english_vowels = {"a", "e", "i", "o", "u", "y"}  
2  
3 print(english_vowels.remove("y"))  
4 print(english_vowels)
```

---

`None`

`{'o', 'a', 'e', 'i', 'u'}`

## 6. Methods that return a new set

# copy()

Same principle as copying a list.

---

```
1 english_vowels_original = {"a", "e", "i", "o", "u"}
2 english_vowels = english_vowels_original.copy()
3
4 english_vowels.add("y")
5
6 print(english_vowels)
7 print(english_vowels_original)
```

---

`{'o', 'y', 'u', 'a', 'i', 'e'}`

`{'o', 'u', 'a', 'i', 'e'}`

# intersection()

= must be included in both sets

---

```
1 sov_languages = {"Japanese", "Korean", "Turkish"}
2 e_asian_languages = {"Japanese", "Korean", "Chinese"}
3
4 e_asian_sov = sov_languages.intersection(e_asian_languages)
5 print(e_asian_sov)
6 #or
7 e_asian_sov = e_asian_languages.intersection(sov_languages)
8 print(e_asian_sov)
9 #or
10 e_asian_sov = e_asian_languages & sov_languages
11 print(e_asian_sov)
```

---

{'Korean', 'Japanese'}

{'Korean', 'Japanese'}

{'Korean', 'Japanese'}

# union()

= take everything from both sets

---

```
1 sov_languages = {"Japanese", "Korean", "Turkish"}
2 e_asian_languages = {"Japanese", "Korean", "Chinese"}
3
4 all_lang_studied = sov_languages.union(e_asian_languages)
5 print(all_lang_studied)
6 #or
7 all_lang_studied = e_asian_languages.union(sov_languages)
8 print(all_lang_studied)
9 #or
10 all_lang_studied = e_asian_languages | sov_languages
11 print(all_lang_studied)
```

---

```
{'Japanese', 'Turkish', 'Chinese', 'Korean'}
{'Japanese', 'Chinese', 'Turkish', 'Korean'}
{'Japanese', 'Chinese', 'Turkish', 'Korean'}
```

-

Minus

(There is also the method `difference()`)

---

```
1 sov_languages = {"Japanese", "Korean", "Latin"}
2 e_asian_languages = {"Japanese", "Korean", "Chinese"}
3
4 sov_not_asian = sov_languages - e_asian_languages
5 print(sov_not_asian)
6
7 asian_not_sov = e_asian_languages - sov_languages
8 print(asian_not_sov)
```

---

{'Latin'}

{'Chinese'}

7. What is allowed in a set?

# Lists not allowed

The elements you add to a set need to be **immutable**.

---

```
1 my_set = set()
2
3 my_set.add("a")
4 my_set.add(5)
5 my_set.add(2.4)
6 my_set.add(("red", "green", "blue"))
7
8 print(my_set)
9
10 my_set.add(["a", "b", "c"]) # DOES NOT WORK!!!!
```

---

```
{'a', 2.4, ('red', 'green', 'blue'), 5}
```

```
TypeError: unhashable type: 'list'
```